Optimising Neural Network Controllers Using Evolutionary Algorithms for the Lunar Lander Problem

40614575

ABSTRACT

This report paper discusses the application of evolutionary algorithms (EAs) to optimize the neural network controller for the Lunar Lander problem. A feed-forward neural network is optimized by employing a population-based optimization technique, wherein Gaussian mutation and tournament selection are applied. Several experiments were run to analyze the effects of various parameters on the algorithm such as mutation scale factor, population size, and number of hidden neurons. The analysis reveals that an increase in hidden neurons increases fitness, but larger populations decrease the speed of optimization when there is a fixed number of evaluations. The optimum results yielded a mean fitness value of 0.175 for 10 runs. The findings highlight the importance of parameter tuning and demonstrate the effectiveness of neuro-evolution for control problems.

1. INTRODUCTION

Evolutionary algorithms (EAs) are probabilistic optimization approaches based on principles of evolution and geneticsely. They are commonly applied to solve complex optimization problems where gradient-based methods are either inefficient or infeasible (Mitchell, 1998; Goldberg & Holland, 1988). One such domain is control systems, where the mapping between inputs and outputs is often non-linear and highly dynamic.

Lunar Lander is one of the hard control tasks that require careful fine-tuning to ensure successful landing. Classical optimization procedures can face difficulties because the problem is non-differentiable. Neuro-evolution, which applies genetic optimization techniques to neural networks, offers a viable solution (Yao, 1999; Floreano et al, 2008).

This paper explores the application of an evolutionary algorithm to optimize the weights' tuning of the neural network controllers in the Lunar Lander task. The paper aims to investigate how different settings affect the process and its results.

2. Approach

2.1 Evolutionary Algorithm Design

The evolutionary algorithm was designed using the generational approach that includes selection, crossover, mutation, and replacement operations (Whitley, 1994).

Tournament selection was adopted because of its simplicity and capability to strike a balance between exploration and exploitation (Eiben & Smith, 2015). Crossover was performed using the uniform method to facilitate genetic exchange between two parents and increase population diversity.

Gaussian mutation was employed as the primary variation operation. Unlike uniform mutation, it allows small incremental adjustments in weight values, which is very helpful when fine-tuning neural networks.

2.2 Neural Network Architecture

The neural network structure employed was a feedforward neural network that had one hidden layer. The network accepts the environmental state variables as inputs and produces actions for controlling the environment.

Three different sizes of hidden layers, including 5, 10, and 15 nodes were tested to evaluate their impact on performance. Increasing the number of hidden nodes increases the network's representational capacity but also increases the search space complexity. This approach aligns with established neuro-evolution techniques that evolve neural network structures and weights (Stanley & Miikkulainen, 2002).

2.3 Parameter Settings

The following parameters were set for the final implementation: -

- Population size: 100
- Maximum evaluations: 20,000
- Mutation rate: 0.05
- Mutation change: 0.1
- Hidden nodes: 15

These parameters were selected based on experimental tuning and prior research on evolutionary algorithms.

3. Experiments and Analysis

3.1 Experimental Setup

Each experiment was repeated 10 times, accounting for the randomness introduced by the evolutionary approach. The performance was measured via the fitness value both on training and test sets.

3.2 Effect of Hidden Nodes

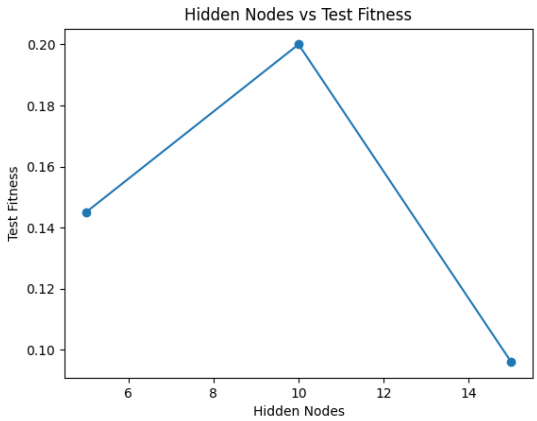

Figure 1 Hidden Nodes vs Test Fitness

Figure 1 demonstrates the effect of hidden layer size on test fitness. As can be seen from the graph, performance significantly improves when the number of hidden nodes goes up from 5 to 15.

The reason behind such improvement is increased representation power of the neural network and ability to approximate the optimal control policy more accurately.

3.3 Population Size Impact

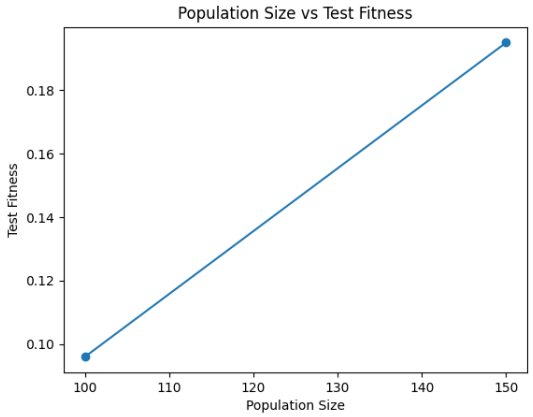

Figure 2 Population Size vs Test Fitness

Figure 2 shows the effect of population size on experimental performance. Surprisingly enough, when the population size increases from 100 to 150, the performance deteriorates. Such result is dictated by the evaluation budget being fixed, which means that only a limited number of generations can be performed. Large populations allow for fewer generations, which restricts further evolution.

3.4 Experiment Performance Analysis

Figure 3 Training vs Test Fitness Across Runs

Figure 3 provides training and test fitness of 10 experimental runs. As can be observed, there is noticeable variation in fitness values due to stochastic nature of the algorithm. Despite some runs demonstrating very low fitness value for training set, the corresponding test fitness does not always show satisfactory performance due to overfitting.

3.5 Critical Analysis

The experiment highlights several prominent features of the evolutionary algorithm. The addition of more hidden nodes improved the algorithm's performance because it allowed the neural network to learn a more complicated behavior. However, on the other hand, the addition of more nodes increases the complexity of the search space and makes optimization more difficult.

In terms of population size, there seems to be a trade-off. While generally larger population sizes should improve diversity, the limited number of evaluations per run means that fewer iterations can occur, thus reducing the ability of the algorithm to optimize the solution. Therefore, in this case, smaller population sizes performed better.

Additionally, the variance between runs highlights the randomness of the evolutionary algorithm. Some runs had relatively high training fitness but very low-test fitness, indicating possible overfitting. Thus, multiple runs are necessary to evaluate the effectiveness of the algorithm.

4. CONCLUSION

This research evaluates the impact of using an evolutionary algorithm on optimizing the neural network controller for the Lunar Lander problem. The outcomes show that the performance of evolutionary algorithms depends significantly on their parameters and the structure of the neural network controller.

A set of 10 experiments were performed to evaluate the effect of parameters such as the size of the hidden layer, population size, and mutation rate. An increase in the number of hidden neurons from 5 to 15 dramatically improved performance by increasing the ability of the neural network to model more sophisticated controllers. However, this change increased the dimensionality of the search space and, therefore, made the optimization process more complicated.

On the contrary, increasing the size of the population from 100 to 150 decreased performance. This could be explained by the fact that only 20,000 evaluations were allowed for the optimization process; hence, more generations were required to evolve larger populations of individuals.

The last configuration was tested in ten separate runs to minimize the variance caused by randomness. The outcomes of the experiments are presented in Table 1.

Metric	Value
Mean Training Fitness	0.06597
Mean Test Fitness	0.17520
Population Size	100

Hidden Nodes	15
Mutation Rate	0.05
Mutation Change	0.1
Activation Function	Sigmoid

From the data collected, there was a noticeable discrepancy between training and testing fitness, implying that overfitting took place during the evolution process. While some trials yielded a very low fitness value from training, this was not always associated with a better performance on the test fitness. It highlights the significance of evaluating the algorithm by considering the performance of several trials.

Generally, from the experiments done using the evolutionary algorithm, it can be noted that the algorithm could successfully yield an effective neural network controller when parameters were tuned well. On the other hand, the varying results from the runs indicate that the algorithm can be stochastic.

To conclude, this study shows that evolutionary algorithms are a good platform for optimizing neural networks for complex controls. However, getting satisfactory results requires a good understanding of how to tune the parameters and designing the neural network appropriately.

5. Future Work

The algorithm presented performed adequately well, however, there were various aspects that could be explored to further increase both efficiency and generalization capacity. For one, the algorithm could use more advanced evolutionary algorithms, such as adaptive mutation rates. Instead of using static mutation rates and step sizes, the algorithm would evolve with varying mutation rates and step sizes.

Another area of interest was the selection and replacement process. Different forms of selection, such as rank-based or fitness proportionate selections, might exert different levels of pressure on the population. Additionally, using different replacement methods, such as generation replacement, might help in maintaining greater diversity within the population.

Another aspect of the algorithm was the usage of different neuro-evolution algorithms, specifically algorithms based on NEAT, where the network topology evolved instead of having a fixed number of neurons. This approach would allow the algorithm to automatically discover a more efficient network structure.

Finally, increasing the number of evaluations or the population size would increase the algorithm's efficiency, but the increase would come at the cost of additional computational resources.

6. REFERENCES

Eiben, A. E., & Smith, J. E. (2015). *Introduction to evolutionary computing* (2nd ed.). Springer. https://doi.org/10.1007/978-3-662-44874-8

Floreano, D., Dürr, P., & Mattiussi, C. (2008). Neuroevolution: From architectures to learning. *Evolutionary Intelligence, 1*(1), 47–62. https://doi.org/10.1007/s12065-007-0002-4

Goldberg, D.E., Holland, J.H. Genetic Algorithms and Machine Learning. *Machine Learning* **3**, 95–99 (1988). https://doi.org/10.1023/A:1022602019183

Mitchell, M. (1998). *An introduction to genetic algorithms.* MIT Press. https://api.semanticscholar.org/CorpusID:2586905

Stanley, K. O., & Miikkulainen, R. (2002). Evolving neural networks through augmenting topologies. *Evolutionary Computation, 10*(2), 99–127. https://doi.org/10.1162/106365602320169811

Whitley, D. (1994). A genetic algorithm tutorial. *Statistics and Computing, 4*(2), 65–85. https://doi.org/10.1007/BF00175354

Yao, X. (1999). Evolving artificial neural networks. *Proceedings of the IEEE, 87*(9), 1423–1447. https://doi.org/10.1109/5.784219